

My Project

Generated by Doxygen 1.17.0

1 Objektinis-programavimas	1
1.1 V1.1 versija	1
1.2 V1.2 versija	2
1.3 V1.5 versija	3
1.4 V2.0 versija	3
1.5 Naudojimo instrukcijos	3
2 Hierarchical Index	5
2.1 Class Hierarchy	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Class Documentation	11
5.1 Studentas Class Reference	11
5.1.1 Detailed Description	12
5.1.2 Constructor & Destructor Documentation	12
5.1.2.1 Studentas() [1/4]	12
5.1.2.2 Studentas() [2/4]	13
5.1.2.3 Studentas() [3/4]	13
5.1.2.4 Studentas() [4/4]	13
5.1.2.5 ~Studentas()	13
5.1.3 Member Function Documentation	13
5.1.3.1 Clear()	13
5.1.3.2 Galutinis()	14
5.1.3.3 Galutinis_mediana()	14
5.1.3.4 getEgzaminas() [1/2]	14
5.1.3.5 getEgzaminas() [2/2]	14
5.1.3.6 getGalutinis() [1/2]	14
5.1.3.7 getGalutinis() [2/2]	14
5.1.3.8 getGalutinis_mediana() [1/2]	14
5.1.3.9 getGalutinis_mediana() [2/2]	15
5.1.3.10 operator=() [1/2]	15
5.1.3.11 operator=() [2/2]	15
5.1.3.12 operator==()	15
5.1.3.13 Spausdinti()	15
5.1.4 Friends And Related Symbol Documentation	15
5.1.4.1 operator<<	15
5.1.4.2 operator>>	16
5.2 Zmogus Class Reference	16
5.2.1 Detailed Description	17

5.2.2 Constructor & Destructor Documentation	17
5.2.2.1 Zmogus() [1/4]	17
5.2.2.2 Zmogus() [2/4]	17
5.2.2.3 Zmogus() [3/4]	17
5.2.2.4 Zmogus() [4/4]	17
5.2.2.5 ~Zmogus()	17
5.2.3 Member Function Documentation	18
5.2.3.1 getPavarde() [1/2]	18
5.2.3.2 getPavarde() [2/2]	18
5.2.3.3 getVardas() [1/2]	18
5.2.3.4 getVardas() [2/2]	18
5.2.3.5 operator=() [1/2]	18
5.2.3.6 operator=() [2/2]	18
5.2.3.7 setPavarde()	19
5.2.3.8 setVardas()	19
5.2.3.9 Spausdinti()	19
6 File Documentation	21
6.1 funkcijos.cpp File Reference	21
6.1.1 Function Documentation	21
6.1.1.1 did_gal_med()	21
6.1.1.2 did_gal_vid()	22
6.1.1.3 did_pav()	22
6.1.1.4 did_var()	22
6.1.1.5 failu_generavimas()	22
6.1.1.6 maz_gal_med()	22
6.1.1.7 maz_gal_vid()	22
6.1.1.8 maz_pav()	23
6.1.1.9 maz_var()	23
6.1.1.10 raides()	23
6.1.1.11 rikiuoti() [1/3]	23
6.1.1.12 rikiuoti() [2/3]	23
6.1.1.13 rikiuoti() [3/3]	23
6.1.1.14 testas_1()	24
6.2 funkcijos.cpp	24
6.3 funkcijos.h File Reference	27
6.3.1 Function Documentation	28
6.3.1.1 dalinimas_i_kategorijas_1()	28
6.3.1.2 dalinimas_i_kategorijas_2()	28
6.3.1.3 dalinimas_i_kategorijas_3()	28
6.3.1.4 dar_vienas_testas()	28
6.3.1.5 did_gal_med()	29

6.3.1.6 did_gal_vid()	29
6.3.1.7 did_pav()	29
6.3.1.8 did_var()	29
6.3.1.9 failu_generavimas()	29
6.3.1.10 generuoti_viska()	29
6.3.1.11 konstruktoriu_testas()	29
6.3.1.12 maz_gal_med()	30
6.3.1.13 maz_gal_vid()	30
6.3.1.14 maz_pav()	30
6.3.1.15 maz_var()	30
6.3.1.16 pasirinkimas()	30
6.3.1.17 pazymiu_generavimas()	30
6.3.1.18 raides()	31
6.3.1.19 rikiavimas()	31
6.3.1.20 rikiuoti() [1/3]	31
6.3.1.21 rikiuoti() [2/3]	31
6.3.1.22 rikiuoti() [3/3]	31
6.3.1.23 skaiciavimai()	31
6.3.1.24 skaitymas()	32
6.3.1.25 skaitymas_is_failo()	32
6.3.1.26 spausdinimas()	32
6.3.1.27 spausdinimas_i_ekrana()	32
6.3.1.28 spausdinimas_i_faila()	32
6.3.1.29 testas_1()	32
6.3.1.30 testas_2()	32
6.3.1.31 testas_3()	33
6.3.1.32 testuoti()	33
6.3.1.33 vector_list_deque()	33
6.4 funkcijos.h	33
6.5 main.cpp File Reference	34
6.5.1 Function Documentation	34
6.5.1.1 main()	34
6.6 main.cpp	34
6.7 mylib.h File Reference	36
6.8 mylib.h	37
6.9 README.md File Reference	40
6.10 testai.cpp File Reference	40
6.10.1 Macro Definition Documentation	40
6.10.1.1 CATCH_CONFIG_RUNNER	40
6.10.2 Function Documentation	40
6.10.2.1 TEST_CASE() [1/10]	40
6.10.2.2 TEST_CASE() [2/10]	41

6.10.2.3 TEST_CASE() [3/10]	41
6.10.2.4 TEST_CASE() [4/10]	41
6.10.2.5 TEST_CASE() [5/10]	41
6.10.2.6 TEST_CASE() [6/10]	41
6.10.2.7 TEST_CASE() [7/10]	41
6.10.2.8 TEST_CASE() [8/10]	42
6.10.2.9 TEST_CASE() [9/10]	42
6.10.2.10 TEST_CASE() [10/10]	42
6.10.2.11 testuoti()	42
6.11 testai.cpp	42
Index	45

Chapter 1

Objektinis-programavimas

1.1 V1.1 versija

Kiekvienai lentelei imami 3 testų duomenys ir pateikiamas jų vidurkis, matuojamas sekundėmis. Testui atlikti buvo naudoti **vector** tipo konteineriai, atlikti tyrimai su skirtingomis optimizavimo vėliavėlėmis - **o1**, **o2** ir **o3**

Klasių analizė

Failų dydžiai	100000 dydžio failas	1000000 dydžio failas
Failo nuskaitymas	0,178971333	1,711526667
Rikiavimas	0,247536	3,258963333
Skirstymas į 2 kategorijas	0,0115029	0,118587333
Spausdinimas	0,149683	1,124083333
Iš viso	0,590147	6,214986667

Struktūrų analizė

Failų dydžiai	100000 dydžio failas	1000000 dydžio failas
Failo nuskaitymas	0,153855667	3,47111
Rikiavimas	0,0717167	0,896167333
Skirstymas į 2 kategorijas	0,013489367	0,237404333
Spausdinimas	0,122633667	1,14252
Iš viso	0,363875333	5,74894

Optimizavimo vėliavėlių analizė

Matuojamas tik bendras programos veikimo laikas

100 000 dydžio failas

Implementacija	Build flag	Testo vykdymo laikas	.exe failo dydis
CLASS	O1	0,453891	347 KB

Implementacija	Build flag	Testo vykdymo laikas	.exe failo dydis
CLASS	O2	0,460904667	327 KB
CLASS	O3	0,470426	352 KB
STRUCT	O1	0,402349667	352 KB
STRUCT	O2	0,378048667	319 KB
STRUCT	O3	0,387034667	334 KB

1 000 000 dydžio failas

Implementacija	Build flag	Testo vykdymo laikas	.exe failo dydis
CLASS	O1	4,51037	347 KB
CLASS	O2	4,501266667	327 KB
CLASS	O3	4,508686667	352 KB
STRUCT	O1	5,888936667	352 KB
STRUCT	O2	5,683196667	319 KB
STRUCT	O3	5,554806667	334 KB

Matome, jog be optimizavimo vėliavėlių programos su klasėmis veikdavo truputį lėčiau negu su struktūromis, tačiau su optimizavimo vėliavėlėmis didesnės apimties failai apdorojami greičiau pasitelkiant klases.

1.2 V1.2 versija

Metodas	Paskirtis
Default konstruktorius	Automatiškai priskiria reikšmes
Konstruktorius su parametrais	Priskiria specifines reikšmes
Getteriai	Grąžina tam tikro vieno parametro reikšmę
Kopijavimo konstruktorius	Nukopijuoja visas tam tikro objekto reikšmes
Kopijavimo priskyrimas	Nukopijuoja visas tam tikro objekto reikšmes (skirtumas toks, kad jau yra sukurtas default objektas, kuriam priskiriamos kito objekto reikšmės pasitelkiant = ženklą)
Move konstruktorius	Perkelia visas vieno objekto reikšmes į kitą objektą ir ištrina pirmojo reikšmes
Move priskyrimas	Perkelia visas vieno objekto reikšmes į kitą objektą ir ištrina pirmojo reikšmes (skirtumas toks, jog jau yra sukurtas default objektas, kuriam priskiriamos kito objekto reikšmės pasitelkiant = ženklą)
Destruktorius	Ištrina objektui priskirtas reikšmes
Įvesties operatorius	Leidžia iš pasirinkto srauto nuskaityti vieno objekto duomenis
Išvesties operatorius	Leidžia į pasirinktą srautą įrašyti vieno objekto duomenis

Perdengtas įvedimo operatorius operator>> turi nuskaityti duomenis iš srauto ir užpildyti objektą.

Perdengtas išvesties operatorius operator<< turi gražiai suformatuoti studento informaciją į tekstinį srautą.

1.3 V1.5 versija

Sukurta bazinė klasė **Zmogus**, iš kurios išvesta klasė **Studentas**.

Klasė **Zmogus** yra abstrakti - tai reiškia, jog jos objektų kūrimas yra negalimas, galima kurti tik jos derived klasių objektus, šiuo atveju, **Studentas** klasės objektus.

Visi v1.2 versijoje realizuoti testai veikia su dabartine derived klase **Studentas**.

1.4 V2.0 versija

Realizuoti pilnai veikiantys Unit (Catch) testai

1.5 Naudojimo instrukcijos

1. Įsitikinkite, jog jūsų įrenginyje yra įdiegta CMake

- Atsidarykite Command Prompt arba PowerShell
- Įveskite: **cmake --version**
- Jei terminalas parodo versiją, viskas gerai. Jei ne, reikia įsidiegti CMake

1. Raskite **run.bat** failą (jis turi būti pagrindiniame projekto kataloge)

2. Paleiskite **run.bat** failą

- *1 variantas*: dukart spustelėkite failą
- *2 variantas*: atidarykite PowerShell arba Command Prompt, nueikite į projekto katalogą ir įveskite ****.\run.bat****

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	16
Studentas	11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	11
Zmogus	16

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

funkcijos.cpp	21
funkcijos.h	27
main.cpp	34
mylib.h	36
testai.cpp	40

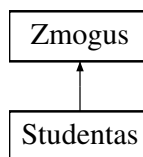
Chapter 5

Class Documentation

5.1 Studentas Class Reference

```
#include <mylib.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
Default konstruktorius.
- [Studentas](#) (string A, string B, vector< int >C, int D)
Konstruktorius su parametrais.
- int [getEgzaminas](#) () const
- int [getEgzaminas](#) ()
- double [getGalutinis](#) () const
- double [getGalutinis](#) ()
- double [getGalutinis_mediana](#) () const
- double [getGalutinis_mediana](#) ()
- [Studentas](#) (const [Studentas](#) &s)
Kopijavimo konstruktorius.
- [Studentas](#) ([Studentas](#) &&s)
Move konstruktorius.
- [Studentas](#) & [operator=](#) (const [Studentas](#) &s)
Kopijavimo priskyrimas.
- [Studentas](#) & [operator=](#) ([Studentas](#) &&s)
Move priskyrimas.
- void [Spausdinti](#) () const override
- double [Galutinis](#) ()
Funkcija, apskaičiuojanti galutinį studento balą pagal vidurkį

- double [Galutinis_mediana](#) ()
Funkcija, apskaičiuojanti galutinį studento balą pagal medianą
- [~Studentas](#) ()
Destruktorius.
- bool [operator==](#) (const [Studentas](#) A) const
Metodas, palyginantis du studentus, ir grąžinantis 'true', jei jų duomenys sutampa, ir 'false', jei nesutampa.
- bool [Clear](#) ()
Metodas, patikrinantis, ar visi duomenys buvo ištrinti.

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()
Default konstruktorius.
- [Zmogus](#) (string A, string B)
Konstruktorius su parametrais.
- string [getVardas](#) () const
- string [getVardas](#) ()
- void [setVardas](#) (const string &v)
- string [getPavarde](#) () const
- string [getPavarde](#) ()
- void [setPavarde](#) (const string &p)
- [Zmogus](#) (const [Zmogus](#) &s)
Kopijavimo konstruktorius.
- [Zmogus](#) ([Zmogus](#) &&s)
Move konstruktorius.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &s)
Kopijavimo priskyrimas.
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&s)
Move priskyrimas.
- virtual [~Zmogus](#) ()
Destruktorius.

Friends

- std::istream & [operator>>](#) (std::istream &in, [Studentas](#) &A)
Ivesties metodas vieno žmogaus duomenims nuskaityti.
- std::ostream & [operator<<](#) (std::ostream &out, const [Studentas](#) &A)
Išvesties metodas vieno žmogaus duomenims išspausdinti.

5.1.1 Detailed Description

Definition at line 114 of file [mylib.h](#).

5.1.2 Constructor & Destructor Documentation

5.1.2.1 Studentas() [1/4]

```
Studentas::Studentas () [inline]
```

Default konstruktorius.

Definition at line 123 of file [mylib.h](#).

5.1.2.2 Studentas() [2/4]

```
Studentas::Studentas (
    string A,
    string B,
    vector< int > C,
    int D) [inline]
```

Konstruktoriaus su parametrais.

Definition at line 130 of file [mylib.h](#).

5.1.2.3 Studentas() [3/4]

```
Studentas::Studentas (
    const Studentas & s) [inline]
```

Kopijavimo konstruktorius.

Definition at line 151 of file [mylib.h](#).

5.1.2.4 Studentas() [4/4]

```
Studentas::Studentas (
    Studentas && s) [inline]
```

Move konstruktorius.

Definition at line 153 of file [mylib.h](#).

5.1.2.5 ~Studentas()

```
Studentas::~~Studentas () [inline]
```

Destruktorius.

Definition at line 236 of file [mylib.h](#).

5.1.3 Member Function Documentation

5.1.3.1 Clear()

```
bool Studentas::Clear () [inline]
```

Metodas, patikrinantis, ar visi duomenys buvo ištrinti.

Definition at line 251 of file [mylib.h](#).

5.1.3.2 Galutinis()

```
double Studentas::Galutinis () [inline]
```

Funkcija, apskaičiuojanti galutinį studento balą pagal vidurkį

Definition at line 223 of file [mylib.h](#).

5.1.3.3 Galutinis_mediana()

```
double Studentas::Galutinis_mediana () [inline]
```

Funkcija, apskaičiuojanti galutinį studento balą pagal medianą

Definition at line 228 of file [mylib.h](#).

5.1.3.4 getEgzaminas() [1/2]

```
int Studentas::getEgzaminas () [inline]
```

Definition at line 139 of file [mylib.h](#).

5.1.3.5 getEgzaminas() [2/2]

```
int Studentas::getEgzaminas () const [inline]
```

Definition at line 135 of file [mylib.h](#).

5.1.3.6 getGalutinis() [1/2]

```
double Studentas::getGalutinis () [inline]
```

Definition at line 144 of file [mylib.h](#).

5.1.3.7 getGalutinis() [2/2]

```
double Studentas::getGalutinis () const [inline]
```

Definition at line 140 of file [mylib.h](#).

5.1.3.8 getGalutinis_mediana() [1/2]

```
double Studentas::getGalutinis_mediana () [inline]
```

Definition at line 149 of file [mylib.h](#).

5.1.3.9 getGalutinis_mediana() [2/2]

```
double Studentas::getGalutinis_mediana () const [inline]
```

Definition at line 145 of file [mylib.h](#).

5.1.3.10 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & s) [inline]
```

Kopijavimo priskyrimas.

Definition at line 161 of file [mylib.h](#).

5.1.3.11 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && s) [inline]
```

Move priskyrimas.

Definition at line 174 of file [mylib.h](#).

5.1.3.12 operator==()

```
bool Studentas::operator== (
    const Studentas A) const [inline]
```

Metodas, palyginantis du studentus, ir grąžinantis 'true', jei jų duomenys sutampa, ir 'false', jei nesutampa.

Definition at line 244 of file [mylib.h](#).

5.1.3.13 Spausdinti()

```
void Studentas::Spausdinti () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

Definition at line 191 of file [mylib.h](#).

5.1.4 Friends And Related Symbol Documentation

5.1.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & A) [friend]
```

Išvesties metodas vieno žmogaus duomenims išspausdinti.

Definition at line 217 of file [mylib.h](#).

5.1.4.2 operator>>

```
std::istream & operator>> (
    std::istream & in,
    Studentas & A) [friend]
```

Ivesties metodus vieno žmogaus duomenims nuskaityti.

Definition at line 196 of file [mylib.h](#).

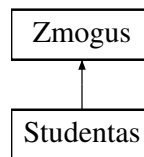
The documentation for this class was generated from the following file:

- [mylib.h](#)

5.2 Zmogus Class Reference

```
#include <mylib.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) ()
Default konstruktorius.
- [Zmogus](#) (string A, string B)
Konstruktorius su parametrais.
- string [getVardas](#) () const
- string [getVardas](#) ()
- void [setVardas](#) (const string &v)
- string [getPavarde](#) () const
- string [getPavarde](#) ()
- void [setPavarde](#) (const string &p)
- [Zmogus](#) (const [Zmogus](#) &s)
Kopijavimo konstruktorius.
- [Zmogus](#) ([Zmogus](#) &&s)
Move konstruktorius.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &s)
Kopijavimo priskyrimas.
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&s)
Move priskyrimas.
- virtual void [Spausdinti](#) () const =0
- virtual [~Zmogus](#) ()
Destruktorius.

5.2.1 Detailed Description

Definition at line 50 of file [mylib.h](#).

5.2.2 Constructor & Destructor Documentation

5.2.2.1 Zmogus() [1/4]

```
Zmogus::Zmogus () [inline]
```

Default konstruktorius.

Definition at line 57 of file [mylib.h](#).

5.2.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (  
    string A,  
    string B) [inline]
```

Konstruktorius su parametrais.

Definition at line 63 of file [mylib.h](#).

5.2.2.3 Zmogus() [3/4]

```
Zmogus::Zmogus (  
    const Zmogus & s) [inline]
```

Kopijavimo konstruktorius.

Definition at line 77 of file [mylib.h](#).

5.2.2.4 Zmogus() [4/4]

```
Zmogus::Zmogus (  
    Zmogus && s) [inline]
```

Move konstruktorius.

Definition at line 79 of file [mylib.h](#).

5.2.2.5 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

Destruktorius.

Definition at line 107 of file [mylib.h](#).

5.2.3 Member Function Documentation

5.2.3.1 getPavarde() [1/2]

```
string Zmogus::getPavarde () [inline]
```

Definition at line 74 of file [mylib.h](#).

5.2.3.2 getPavarde() [2/2]

```
string Zmogus::getPavarde () const [inline]
```

Definition at line 70 of file [mylib.h](#).

5.2.3.3 getVardas() [1/2]

```
string Zmogus::getVardas () [inline]
```

Definition at line 68 of file [mylib.h](#).

5.2.3.4 getVardas() [2/2]

```
string Zmogus::getVardas () const [inline]
```

Definition at line 64 of file [mylib.h](#).

5.2.3.5 operator=() [1/2]

```
Zmogus & Zmogus::operator= (  
    const Zmogus & s) [inline]
```

Kopijavimo priskyrimas.

Definition at line 85 of file [mylib.h](#).

5.2.3.6 operator=() [2/2]

```
Zmogus & Zmogus::operator= (  
    Zmogus && s) [inline]
```

Move priskyrimas.

Definition at line 94 of file [mylib.h](#).

5.2.3.7 setPavarde()

```
void Zmogus::setPavarde (  
    const string & p) [inline]
```

Definition at line 75 of file [mylib.h](#).

5.2.3.8 setVardas()

```
void Zmogus::setVardas (  
    const string & v) [inline]
```

Definition at line 69 of file [mylib.h](#).

5.2.3.9 Spausdinti()

```
virtual void Zmogus::Spausdinti () const [pure virtual]
```

Implemented in [Studentas](#).

The documentation for this class was generated from the following file:

- [mylib.h](#)

Chapter 6

File Documentation

6.1 funkcijos.cpp File Reference

```
#include "funkcijos.h"
```

Functions

- char [raides](#) (char a, char b)
- bool [did_var](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [maz_var](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [did_pav](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [maz_pav](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [did_gal_med](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [maz_gal_med](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [did_gal_vid](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [maz_gal_vid](#) ([Studentas](#) &A, [Studentas](#) &B)
- void [rikiuoti](#) (vector< [Studentas](#) > &studentai, char tvarka, string b)
- void [rikiuoti](#) (deque< [Studentas](#) > &studentai, char tvarka, string b)
- void [rikiuoti](#) (list< [Studentas](#) > &studentai, char tvarka, string b)
- void [failu_generavimas](#) (int n)
- void [testas_1](#) (int n)

6.1.1 Function Documentation

6.1.1.1 did_gal_med()

```
bool did_gal_med (  
    Studentas & A,  
    Studentas & B)
```

Definition at line [58](#) of file [funkcijos.cpp](#).

6.1.1.2 did_gal_vid()

```
bool did_gal_vid (  
    Studentas & A,  
    Studentas & B)
```

Definition at line 66 of file [funkcijos.cpp](#).

6.1.1.3 did_pav()

```
bool did_pav (  
    Studentas & A,  
    Studentas & B)
```

Definition at line 50 of file [funkcijos.cpp](#).

6.1.1.4 did_var()

```
bool did_var (  
    Studentas & A,  
    Studentas & B)
```

Definition at line 42 of file [funkcijos.cpp](#).

6.1.1.5 failu_generavimas()

```
void failu_generavimas (  
    int n)
```

Definition at line 143 of file [funkcijos.cpp](#).

6.1.1.6 maz_gal_med()

```
bool maz_gal_med (  
    Studentas & A,  
    Studentas & B)
```

Definition at line 62 of file [funkcijos.cpp](#).

6.1.1.7 maz_gal_vid()

```
bool maz_gal_vid (  
    Studentas & A,  
    Studentas & B)
```

Definition at line 70 of file [funkcijos.cpp](#).

6.1.1.8 maz_pav()

```
bool maz_pav (
    Studentas & A,
    Studentas & B)
```

Definition at line 54 of file [funkcijos.cpp](#).

6.1.1.9 maz_var()

```
bool maz_var (
    Studentas & A,
    Studentas & B)
```

Definition at line 46 of file [funkcijos.cpp](#).

6.1.1.10 raides()

```
char raides (
    char a,
    char b)
```

Definition at line 2 of file [funkcijos.cpp](#).

6.1.1.11 rikiuoti() [1/3]

```
void rikiuoti (
    deque< Studentas > & studentai,
    char tvarka,
    string b)
```

Definition at line 97 of file [funkcijos.cpp](#).

6.1.1.12 rikiuoti() [2/3]

```
void rikiuoti (
    list< Studentas > & studentai,
    char tvarka,
    string b)
```

Definition at line 120 of file [funkcijos.cpp](#).

6.1.1.13 rikiuoti() [3/3]

```
void rikiuoti (
    vector< Studentas > & studentai,
    char tvarka,
    string b)
```

Definition at line 74 of file [funkcijos.cpp](#).

6.1.1.14 testas_1()

```
void testas_1 (
    int n)
```

Definition at line 167 of file [funkcijos.cpp](#).

6.2 funkcijos.cpp

[Go to the documentation of this file.](#)

```
00001 #include "funkcijos.h"
00002 char raides(char a, char b)
00003 {
00004     char c;
00005     while(true)
00006     {
00007         try
00008         {
00009             cin>c;
00010             if(cin.fail() || (c!=a && c!=b))
00011                 throw std::runtime_error(string("Iveskite '") + a + string("' arba '") + b + string("': "));
00012
00013             cin.ignore(1000, '\n');
00014             break;
00015         }
00016         catch(const std::exception& e)
00017         {
00018             cerr<<"Klaida! " << e.what() << endl;
00019             cin.clear();
00020             cin.ignore(1000, '\n');
00021         }
00022     }
00023     return c;
00024 }
00025 // void skaiciavimai(Studentas& A)
00026 // {
00027 //     sort(A.pazymiai.begin(), A.pazymiai.end());
00028 //     if(A.pazymiai.size()==0)
00029 //     {
00030 //         A.galutinis=A.egzaminas*0.6;
00031 //         A.galutinis_mediana=A.egzaminas*0.6;
00032 //     }
00033 //     else
00034 //     {
00035 //         double vid=accumulate(A.pazymiai.begin(), A.pazymiai.end(), 0.0)/A.pazymiai.size();
00036 //         A.galutinis=vid*0.4+A.egzaminas*0.6;
00037 //         if(A.pazymiai.size()%2==1)
00038 //             A.galutinis_mediana=A.pazymiai[A.pazymiai.size()/2]*0.4+A.egzaminas*0.6;
00039 //         else
00040 //             A.galutinis_mediana=(A.pazymiai[A.pazymiai.size()/2]+A.pazymiai[A.pazymiai.size()/2-1])/2.0*0.4+A.egzaminas*0.6;
00041 //     }
00042 bool did_var(Studentas& A, Studentas& B)
00043 {
00044     return A.getVardas()<B.getVardas();
00045 }
00046 bool maz_var(Studentas& A, Studentas& B)
00047 {
00048     return A.getVardas()>B.getVardas();
00049 }
00050 bool did_pav(Studentas& A, Studentas& B)
00051 {
00052     return A.getPavarde()<B.getPavarde();
00053 }
00054 bool maz_pav(Studentas& A, Studentas& B)
00055 {
00056     return A.getPavarde()>B.getPavarde();
00057 }
00058 bool did_gal_med(Studentas& A, Studentas& B)
00059 {
00060     return A.Galutinis_mediana()<B.Galutinis_mediana();
00061 }
00062 bool maz_gal_med(Studentas& A, Studentas& B)
00063 {
00064     return A.Galutinis_mediana()>B.Galutinis_mediana();
00065 }
00066 bool did_gal_vid(Studentas& A, Studentas& B)
```

```

00067 {
00068     return A.Galutinis()<B.Galutinis();
00069 }
00070 bool maz_gal_vid(Studentas& A, Studentas& B)
00071 {
00072     return A.Galutinis()>B.Galutinis();
00073 }
00074 void rikiuoti(vector<Studentas>&studentai, char tvarka, string b)
00075 {
00076     if(tvarka=='d')
00077     {
00078         if(b=="var")
00079             sort(studentai.begin(), studentai.end(), did_var);
00080         else if(b=="pav")
00081             sort(studentai.begin(), studentai.end(), did_pav);
00082         else if(b=="gal_med")
00083             sort(studentai.begin(), studentai.end(), did_gal_med);
00084         else sort(studentai.begin(), studentai.end(), did_gal_vid);
00085     }
00086     else
00087     {
00088         if(b=="var")
00089             sort(studentai.begin(), studentai.end(), maz_var);
00090         else if(b=="pav")
00091             sort(studentai.begin(), studentai.end(), maz_pav);
00092         else if(b=="gal_med")
00093             sort(studentai.begin(), studentai.end(), maz_gal_med);
00094         else sort(studentai.begin(), studentai.end(), maz_gal_vid);
00095     }
00096 }
00097 void rikiuoti(deque<Studentas>&studentai, char tvarka, string b)
00098 {
00099     if(tvarka=='d')
00100     {
00101         if(b=="var")
00102             sort(studentai.begin(), studentai.end(), did_var);
00103         else if(b=="pav")
00104             sort(studentai.begin(), studentai.end(), did_pav);
00105         else if(b=="gal_med")
00106             sort(studentai.begin(), studentai.end(), did_gal_med);
00107         else sort(studentai.begin(), studentai.end(), did_gal_vid);
00108     }
00109     else
00110     {
00111         if(b=="var")
00112             sort(studentai.begin(), studentai.end(), maz_var);
00113         else if(b=="pav")
00114             sort(studentai.begin(), studentai.end(), maz_pav);
00115         else if(b=="gal_med")
00116             sort(studentai.begin(), studentai.end(), maz_gal_med);
00117         else sort(studentai.begin(), studentai.end(), maz_gal_vid);
00118     }
00119 }
00120 void rikiuoti(list<Studentas>&studentai, char tvarka, string b)
00121 {
00122     if(tvarka=='d')
00123     {
00124         if(b=="var")
00125             studentai.sort(did_var);
00126         else if(b=="pav")
00127             studentai.sort(did_pav);
00128         else if(b=="gal_med")
00129             studentai.sort(did_gal_med);
00130         else studentai.sort(did_gal_vid);
00131     }
00132     else
00133     {
00134         if(b=="var")
00135             studentai.sort(maz_var);
00136         else if(b=="pav")
00137             studentai.sort(maz_pav);
00138         else if(b=="gal_med")
00139             studentai.sort(maz_gal_med);
00140         else studentai.sort(maz_gal_vid);
00141     }
00142 }
00143 void failu_generavimas(int n)
00144 {
00145     ofstream f("failas_"+std::to_string(n)+".txt");
00146     int m=rand()%20+1;
00147     ostringstream ss;
00148     ss<<left<setw(21)<<"Vardas"<<setw(21)<<"Pavarde";
00149     for(int i=0; i<m; i++)
00150     {
00151         ss<<setw(5)<<("ND"+std::to_string(i+1))<<" ";
00152     }
00153 }

```

```

00154     ss<<setw(5)<<"Egzaminas"<<endl;
00155     for(int i=0; i<n; i++)
00156     {
00157         ss<<left<<setw(20)<<("Vardas"+std::to_string(i+1))<<" " <<setw(20)<<("Pavarde"+std::to_string(i+1))<<"
";
00158         for(int j=0; j<m; j++)
00159         {
00160             ss<<setw(5)<<rand()%10+1<<" ";
00161         }
00162         ss<<setw(5)<<rand()%10+1<<endl;
00163     }
00164     f<<ss.str();
00165     f.close();
00166 }
00167 void testas_1(int n)
00168 {
00169     auto start = high_resolution_clock::now();
00170     failu_generavimas(n);
00171     auto end = high_resolution_clock::now();
00172     duration<double> laikas=end-start;
00173     cout<<" dydzio faila sugeneruoti uztruko "<<laikas.count()<<" s"<<endl;
00174 }
00175 // void konstruktoriu_testas()
00176 // {
00177 //     Studentas stud;
00178 //     if(stud==Studentas("Vardas", "Pavarde", { }, 0))
00179 //         cout<<"Default konstruktorius veikia"<<endl;
00180 //     else cout<<"Default konstruktorius neveikia"<<endl;
00181 //
00182 //     Studentas Stud("Vardenis", "Pavardenis", {5, 8, 3}, 9);
00183 //     if(Stud==Studentas("Vardenis", "Pavardenis", {5, 8, 3}, 9))
00184 //         cout<<"Konstruktorius su parametrais veikia"<<endl;
00185 //     else cout<<"Konstruktorius su parametrais neveikia"<<endl;
00186 //
00187 //     if(Stud.getVardas()=="Vardenis")
00188 //         cout<<"Vardo 'getteris' veikia"<<endl;
00189 //     else cout<<"Vardo 'getteris' neveikia"<<endl;
00190 //
00191 //     if(Stud.getPavarde()=="Pavardenis")
00192 //         cout<<"Pavardes 'getteris' veikia"<<endl;
00193 //     else cout<<"Pavardes 'getteris' neveikia"<<endl;
00194 //
00195 //     if(Stud.getEgzaminas()==9)
00196 //         cout<<"Egzamino 'getteris' veikia"<<endl;
00197 //     else cout<<"Egzamino 'getteris' neveikia"<<endl;
00198 //
00199 //     if(Stud.getGalutinis()==Stud.Galutinis())
00200 //         cout<<"Galutinio rezultato 'getteris' veikia"<<endl;
00201 //     else cout<<"Galutinio rezultato 'getteris' neveikia"<<endl;
00202 //
00203 //     if(Stud.getGalutinis_mediana()==Stud.Galutinis_mediana())
00204 //         cout<<"Galutinio rezultato pagal mediana 'getteris' veikia"<<endl;
00205 //     else cout<<"Galutinio rezultato pagal mediana 'getteris' neveikia"<<endl;
00206 //
00207 //     Studentas stud2(Stud);
00208 //     if(stud2==Stud)
00209 //         cout<<"Kopijavimo konstruktorius veikia"<<endl;
00210 //     else cout<<"Kopijavimo konstruktorius neveikia"<<endl;
00211 //
00212 //     Studentas stud3;
00213 //     stud3=Stud;
00214 //     if(stud3==Stud)
00215 //         cout<<"Kopijavimo priskyrimas veikia"<<endl;
00216 //     else cout<<"Kopijavimo priskyrimas neveikia"<<endl;
00217 //
00218 //     Studentas Stud2(std::move(Stud));
00219 //     if(Stud2==Studentas("Vardenis", "Pavardenis", {5, 8, 3}, 9) && Stud.Clear()==true)
00220 //         cout<<"Move konstruktorius veikia"<<endl;
00221 //     else cout<<"Move konstruktorius neveikia"<<endl;
00222 //
00223 //     Studentas Stud1("Vardenis", "Pavardenis", {5, 8, 3}, 9);
00224 //     Studentas stud1;
00225 //     stud1=std::move(Stud1);
00226 //     if(stud1==Studentas("Vardenis", "Pavardenis", {5, 8, 3}, 9) && Stud1.Clear()==true)
00227 //         cout<<"Move priskyrimas veikia"<<endl;
00228 //     else cout<<"Move priskyrimas neveikia"<<endl;
00229 //
00230 //     Stud1.~Studentas();
00231 //     if(Stud1.Clear()==true)
00232 //         cout<<"Destruktorius veikia"<<endl;
00233 //     else cout<<"Destruktorius neveikia"<<endl;
00234 //
00235 //     std::istringstream in("Jonas Jonaitis 10 9 8 7");
00236 //     Studentas s;
00237 //     in >> s;
00238 //     if(s.getVardas()=="Jonas" && s.getPavarde()=="Jonaitis")
00239 //         cout<<"Ivesties metodus veikia"<<endl;

```



```

00240 //      else cout<<"Ivesties metodos neveikia"<<endl;
00241
00242 //      Studentas s1("Jonas", "Jonaitis", {10, 9, 8}, 7);
00243 //      std::ostringstream out;
00244 //      out << s1;
00245 //      if(out.str().find("Jonas") != string::npos && out.str().find("Jonaitis") != string::npos)
00246 //          cout<<"Ivesties metodos veikia"<<endl;
00247 //      else cout<<"Ivesties metodos neveikia"<<endl;
00248 //  }

```

6.3 funkcijos.h File Reference

```

#include "mylib.h"
#include "funkcijos.hpp"

```

Functions

- char [raides](#) (char a, char b)
- void [skaiciavimai](#) ([Studentas](#) &A)
- template<typename Konteineris>
void [skaitymas_is_failo](#) (Konteineris &studentai)
- template<typename Konteineris>
void [skaitymas](#) (Konteineris &studentai)
- template<typename Konteineris>
void [pazymiu_generavimas](#) (Konteineris &studentai)
- template<typename Konteineris>
void [generuoti_viska](#) (Konteineris &studentai)
- template<typename Konteineris>
void [spausdinimas_i_ekrana](#) (const Konteineris &studentai)
- template<typename Konteineris>
void [spausdinimas_i_faila](#) (const Konteineris &studentai)
- template<typename Konteineris>
void [pasirinkimas](#) (Konteineris &studentai)
- bool [did_var](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [maz_var](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [did_pav](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [maz_pav](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [did_gal_med](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [maz_gal_med](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [did_gal_vid](#) ([Studentas](#) &A, [Studentas](#) &B)
- bool [maz_gal_vid](#) ([Studentas](#) &A, [Studentas](#) &B)
- void [rikiuoti](#) (vector< [Studentas](#) > &studentai, char tvarka, string b)
- void [rikiuoti](#) (deque< [Studentas](#) > &studentai, char tvarka, string b)
- void [rikiuoti](#) (list< [Studentas](#) > &studentai, char tvarka, string b)
- template<typename Konteineris>
void [rikiavimas](#) (Konteineris &studentai)
- template<typename Konteineris>
void [spausdinimas](#) (const Konteineris &studentai)
- void [failu_generavimas](#) (int n)
- template<typename Konteineris>
void [dalinimas_i_kategorijas_1](#) (Konteineris &studentai, Konteineris &tinginiai, Konteineris &darbstuoliai)
- template<typename Konteineris>
void [dalinimas_i_kategorijas_2](#) (Konteineris &studentai, Konteineris &tinginiai)

- `template<typename Konteineris>`
`void dalinimas_i_kategorijas_3 (Konteineris &studentai, Konteineris &tinginiai, Konteineris &darbstuoliai)`
- `void testas_1 (int n)`
- `template<typename Konteineris>`
`void testas_2 (Konteineris &studentai, int n)`
- `template<typename Konteineris>`
`void vector_list_deque (Konteineris &studentai)`
- `template<typename Konteineris>`
`void testas_3 (Konteineris &studentai, int n)`
- `template<typename Konteineris>`
`void dar_vienas_testas (Konteineris &studentai, int n)`
- `void testuoti ()`
- `void konstruktoriu_testas ()`

6.3.1 Function Documentation

6.3.1.1 dalinimas_i_kategorijas_1()

```
template<typename Konteineris>
void dalinimas_i_kategorijas_1 (
    Konteineris & studentai,
    Konteineris & tinginiai,
    Konteineris & darbstuoliai)
```

6.3.1.2 dalinimas_i_kategorijas_2()

```
template<typename Konteineris>
void dalinimas_i_kategorijas_2 (
    Konteineris & studentai,
    Konteineris & tinginiai)
```

6.3.1.3 dalinimas_i_kategorijas_3()

```
template<typename Konteineris>
void dalinimas_i_kategorijas_3 (
    Konteineris & studentai,
    Konteineris & tinginiai,
    Konteineris & darbstuoliai)
```

6.3.1.4 dar_vienas_testas()

```
template<typename Konteineris>
void dar_vienas_testas (
    Konteineris & studentai,
    int n)
```

6.3.1.5 did_gal_med()

```
bool did_gal_med (  
    Studentas & A,  
    Studentas & B)
```

Definition at line 58 of file [funkcijos.cpp](#).

6.3.1.6 did_gal_vid()

```
bool did_gal_vid (  
    Studentas & A,  
    Studentas & B)
```

Definition at line 66 of file [funkcijos.cpp](#).

6.3.1.7 did_pav()

```
bool did_pav (  
    Studentas & A,  
    Studentas & B)
```

Definition at line 50 of file [funkcijos.cpp](#).

6.3.1.8 did_var()

```
bool did_var (  
    Studentas & A,  
    Studentas & B)
```

Definition at line 42 of file [funkcijos.cpp](#).

6.3.1.9 failu_generavimas()

```
void failu_generavimas (  
    int n)
```

Definition at line 143 of file [funkcijos.cpp](#).

6.3.1.10 generuoti_viska()

```
template<typename Konteineris>  
void generuoti_viska (  
    Konteineris & studentai)
```

6.3.1.11 konstruktoriu_testas()

```
void konstruktoriu_testas ()
```

6.3.1.12 maz_gal_med()

```
bool maz_gal_med (
    Studentas & A,
    Studentas & B)
```

Definition at line 62 of file [funkcijos.cpp](#).

6.3.1.13 maz_gal_vid()

```
bool maz_gal_vid (
    Studentas & A,
    Studentas & B)
```

Definition at line 70 of file [funkcijos.cpp](#).

6.3.1.14 maz_pav()

```
bool maz_pav (
    Studentas & A,
    Studentas & B)
```

Definition at line 54 of file [funkcijos.cpp](#).

6.3.1.15 maz_var()

```
bool maz_var (
    Studentas & A,
    Studentas & B)
```

Definition at line 46 of file [funkcijos.cpp](#).

6.3.1.16 pasirinkimas()

```
template<typename Konteineris>
void pasirinkimas (
    Konteineris & studentai)
```

6.3.1.17 pazymiu_generavimas()

```
template<typename Konteineris>
void pazymiu_generavimas (
    Konteineris & studentai)
```

6.3.1.18 raides()

```
char raides (
    char a,
    char b)
```

Definition at line 2 of file [funkcijos.cpp](#).

6.3.1.19 rikiavimas()

```
template<typename Konteineris>
void rikiavimas (
    Konteineris & studentai)
```

6.3.1.20 rikiuoti() [1/3]

```
void rikiuoti (
    deque< Studentas > & studentai,
    char tvarka,
    string b)
```

Definition at line 97 of file [funkcijos.cpp](#).

6.3.1.21 rikiuoti() [2/3]

```
void rikiuoti (
    list< Studentas > & studentai,
    char tvarka,
    string b)
```

Definition at line 120 of file [funkcijos.cpp](#).

6.3.1.22 rikiuoti() [3/3]

```
void rikiuoti (
    vector< Studentas > & studentai,
    char tvarka,
    string b)
```

Definition at line 74 of file [funkcijos.cpp](#).

6.3.1.23 skaiciavimai()

```
void skaiciavimai (
    Studentas & A)
```

6.3.1.24 skaitymas()

```
template<typename Konteineris>
void skaitymas (
    Konteineris & studentai)
```

6.3.1.25 skaitymas_is_failo()

```
template<typename Konteineris>
void skaitymas_is_failo (
    Konteineris & studentai)
```

6.3.1.26 spausdinimas()

```
template<typename Konteineris>
void spausdinimas (
    const Konteineris & studentai)
```

6.3.1.27 spausdinimas_i_ekrana()

```
template<typename Konteineris>
void spausdinimas_i_ekrana (
    const Konteineris & studentai)
```

6.3.1.28 spausdinimas_i_faila()

```
template<typename Konteineris>
void spausdinimas_i_faila (
    const Konteineris & studentai)
```

6.3.1.29 testas_1()

```
void testas_1 (
    int n)
```

Definition at line [167](#) of file [funkcijos.cpp](#).

6.3.1.30 testas_2()

```
template<typename Konteineris>
void testas_2 (
    Konteineris & studentai,
    int n)
```

6.3.1.31 testas_3()

```
template<typename Konteineris>
void testas_3 (
    Konteineris & studentai,
    int n)
```

6.3.1.32 testuoti()

```
void testuoti ()
```

Definition at line 77 of file [testai.cpp](#).

6.3.1.33 vector_list_deque()

```
template<typename Konteineris>
void vector_list_deque (
    Konteineris & studentai)
```

6.4 funkcijos.h

[Go to the documentation of this file.](#)

```
00001 #include "mylib.h"
00002 char raides(char a, char b);
00003 void skaiciavimai(Studentas& A);
00004 template <typename Konteineris>
00005 void skaitymas_is_failo(Konteineris&studentai);
00006 template <typename Konteineris>
00007 void skaitymas(Konteineris&studentai);
00008 template <typename Konteineris>
00009 void pazymiu_generavimas(Konteineris&studentai);
00010 template <typename Konteineris>
00011 void generuoti_viska(Konteineris&studentai);
00012 template <typename Konteineris>
00013 void spausdinimas_i_ekrana(const Konteineris&studentai);
00014 template <typename Konteineris>
00015 void spausdinimas_i_faila(const Konteineris&studentai);
00016 template <typename Konteineris>
00017 void pasirinkimas(Konteineris&studentai);
00018 bool did_var(Studentas& A, Studentas& B);
00019 bool maz_var(Studentas& A, Studentas& B);
00020 bool did_pav(Studentas& A, Studentas& B);
00021 bool maz_pav(Studentas& A, Studentas& B);
00022 bool did_gal_med(Studentas& A, Studentas& B);
00023 bool maz_gal_med(Studentas& A, Studentas& B);
00024 bool did_gal_vid(Studentas& A, Studentas& B);
00025 bool maz_gal_vid(Studentas& A, Studentas& B);
00026 void rikiuoti(vector<Studentas>&studentai, char tvarka, string b);
00027 void rikiuoti(deque<Studentas>&studentai, char tvarka, string b);
00028 void rikiuoti(list<Studentas>&studentai, char tvarka, string b);
00029 template <typename Konteineris>
00030 void rikiavimas(Konteineris&studentai);
00031 template <typename Konteineris>
00032 void spausdinimas(const Konteineris&studentai);
00033 void failu_generavimas(int n);
00034 template <typename Konteineris>
00035 void dalinimas_i_kategorijas_1(Konteineris&studentai, Konteineris&tinginiai,
    Konteineris&darbstuoliai);
00036 template <typename Konteineris>
00037 void dalinimas_i_kategorijas_2(Konteineris&studentai, Konteineris&tinginiai);
00038 template <typename Konteineris>
00039 void dalinimas_i_kategorijas_3(Konteineris&studentai, Konteineris&tinginiai,
    Konteineris&darbstuoliai);
00040 void testas_1(int n);
00041 template <typename Konteineris>
00042 void testas_2(Konteineris&studentai, int n);
```

```

00043 template <typename Konteineris>
00044 void vector_list_deque(Konteineris& studentai);
00045 template <typename Konteineris>
00046 void testas_3(Konteineris&studentai, int n);
00047 template <typename Konteineris>
00048 void dar_vienas_testas(Konteineris&studentai, int n);
00049 void testuoti();
00050 void konstruktoriu_testas();
00051 #include "funkcijos.hpp"

```

6.5 main.cpp File Reference

```
#include "funkcijos.h"
```

Functions

- `int main()`

6.5.1 Function Documentation

6.5.1.1 main()

```
int main ()
```

Definition at line 2 of file [main.cpp](#).

6.6 main.cpp

[Go to the documentation of this file.](#)

```

00001 #include "funkcijos.h"
00002 int main()
00003 {
00004     srand(static_cast<unsigned>(time(NULL)));
00005
00006     int choice;
00007     cout<<"===Menu==="<<endl;
00008     cout<<"1. Naudoti vektorius"<<endl;
00009     cout<<"2. Naudoti list"<<endl;
00010     cout<<"3. Naudoti deque"<<endl;
00011     cout<<"4. Programos testavimas 1 (failu generavimas)"<<endl;
00012     cout<<"5. Programos testavimas 2 (generuotu failu apdorojimas)"<<endl;
00013     cout<<"6. Programos testavimas 3 (strategiju lyginimas)"<<endl;
00014     cout<<"7. Greiciausios strategijos spartos analize su skirtingais optimizavimo flag'ais"<<endl;
00015     cout<<"8. Testuoti konstruktoriu, kopijavimo, perkeliimo metodus"<<endl;
00016     cout<<"9. Baigti darba"<<endl;
00017     while(true)
00018     {
00019         try
00020         {
00021             cin>>choice;
00022             if(cin.fail() || choice<1 || choice>9)
00023                 throw std::runtime_error("Iveskite skaiciu 1-9: ");
00024
00025             cin.ignore(1000, '\n');
00026             break;
00027         }
00028         catch(const std::exception& e)
00029         {
00030             cerr<<"Klaida! "«e.what()<<endl;
00031             cin.clear();
00032             cin.ignore(1000, '\n');

```



```

00033     }
00034 }
00035 switch(choice)
00036 {
00037     case 1:
00038     {
00039         vector<Studentas> studentai;
00040         vector_list_deque(studentai);
00041         break;
00042     }
00043     case 2:
00044     {
00045         list<Studentas> studentai;
00046         vector_list_deque(studentai);
00047         break;
00048     }
00049     case 3:
00050     {
00051         deque<Studentas> studentai;
00052         vector_list_deque(studentai);
00053         break;
00054     }
00055     case 4:
00056     {
00057         testas_1(1000);
00058         testas_1(10000);
00059         testas_1(100000);
00060         testas_1(1000000);
00061         testas_1(10000000);
00062         exit(0);
00063     }
00064     case 5:
00065     {
00066         vector<Studentas> studentai;
00067         list<Studentas> studentail;
00068         deque<Studentas> studentai2;
00069         cout<<"1000 irasu failai:"<<endl;
00070         cout<<"Vector testas:"<<endl;
00071         testas_2(studentai, 1000);
00072         cout<<"List testas:"<<endl;
00073         testas_2(studentail, 1000);
00074         cout<<"Deque testas:"<<endl;
00075         testas_2(studentai2, 1000);
00076
00077         cout<<"10000 irasu failai:"<<endl;
00078         cout<<"Vector testas:"<<endl;
00079         testas_2(studentai, 10000);
00080         cout<<"List testas:"<<endl;
00081         testas_2(studentail, 10000);
00082         cout<<"Deque testas:"<<endl;
00083         testas_2(studentai2, 10000);
00084
00085         cout<<"100000 irasu failai:"<<endl;
00086         cout<<"Vector testas:"<<endl;
00087         testas_2(studentai, 100000);
00088         cout<<"List testas:"<<endl;
00089         testas_2(studentail, 100000);
00090         cout<<"Deque testas:"<<endl;
00091         testas_2(studentai2, 100000);
00092
00093         cout<<"1000000 irasu failai:"<<endl;
00094         cout<<"Vector testas:"<<endl;
00095         testas_2(studentai, 1000000);
00096         cout<<"List testas:"<<endl;
00097         testas_2(studentail, 1000000);
00098         cout<<"Deque testas:"<<endl;
00099         testas_2(studentai2, 1000000);
00100
00101         cout<<"10000000 irasu failai:"<<endl;
00102         cout<<"Vector testas:"<<endl;
00103         testas_2(studentai, 10000000);
00104         cout<<"List testas:"<<endl;
00105         testas_2(studentail, 10000000);
00106         cout<<"Deque testas:"<<endl;
00107         testas_2(studentai2, 10000000);
00108         exit(0);
00109     }
00110     case 6:
00111     {
00112         vector<Studentas> studentai;
00113         list<Studentas> studentail;
00114         deque<Studentas> studentai2;
00115         cout<<"1000 irasu failai:"<<endl;
00116         cout<<"--Vector--"<<endl;
00117         testas_3(studentai, 1000);
00118         cout<<"--List--"<<endl;
00119         testas_3(studentail, 1000);

```

```

00120         cout<<"--Deque--"<<endl;
00121         testas_3(studentai2, 1000);
00122
00123         cout<<endl<<"10000 irasu failai:"<<endl;
00124         cout<<"--Vector--"<<endl;
00125         testas_3(studentai, 10000);
00126         cout<<"--List--"<<endl;
00127         testas_3(studentai1, 10000);
00128         cout<<"--Deque--"<<endl;
00129         testas_3(studentai2, 10000);
00130
00131         cout<<endl<<"100000 irasu failai:"<<endl;
00132         cout<<"--Vector--"<<endl;
00133         testas_3(studentai, 100000);
00134         cout<<"--List--"<<endl;
00135         testas_3(studentai1, 100000);
00136         cout<<"--Deque--"<<endl;
00137         testas_3(studentai2, 100000);
00138
00139         cout<<endl<<"1000000 irasu failai:"<<endl;
00140         cout<<"--Vector--"<<endl;
00141         testas_3(studentai, 1000000);
00142         cout<<"--List--"<<endl;
00143         testas_3(studentai1, 1000000);
00144         cout<<"--Deque--"<<endl;
00145         testas_3(studentai2, 1000000);
00146
00147         cout<<endl<<"10000000 irasu failai:"<<endl;
00148         cout<<"--Vector--"<<endl;
00149         testas_3(studentai, 10000000);
00150         cout<<"--List--"<<endl;
00151         testas_3(studentai1, 10000000);
00152         cout<<"--Deque--"<<endl;
00153         testas_3(studentai2, 10000000);
00154
00155         break;
00156     }
00157     case 7:
00158     {
00159         vector<Studentas> studentai;
00160         dar_vienas_testas(studentai, 100000);
00161         dar_vienas_testas(studentai, 1000000);
00162     }
00163     case 8:
00164         testuoti();
00165     case 9:
00166         exit(0);
00167 }
00168
00169 return 0;
00170 }

```

6.7 mylib.h File Reference

```

#include <iostream>
#include <ctime>
#include <stdexcept>
#include <exception>
#include <list>
#include <deque>
#include <type_traits>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <iomanip>
#include <algorithm>
#include <cstdlib>
#include <numeric>
#include <chrono>

```

Classes

- class [Zmogus](#)
- class [Studentas](#)

6.8 mylib.h

[Go to the documentation of this file.](#)

```

00001 #ifndef MYLIB_H
00002 #define MYLIB_H
00003 #include <iostream>
00004 #include <ctime>
00005 #include <stdexcept>
00006 #include <exception>
00007 #include <list>
00008 #include <deque>
00009 #include <type_traits>
00010 #include <fstream>
00011 #include <sstream>
00012 #include <string>
00013 #include <vector>
00014 #include <iomanip>
00015 #include <algorithm>
00016 #include <cstdlib>
00017 #include <numeric>
00018 #include <chrono>
00019 using std::cin;
00020 using std::cout;
00021 using std::vector;
00022 using std::list;
00023 using std::deque;
00024 using std::left;
00025 using std::setw;
00026 using std::string;
00027 using std::endl;
00028 using std::fixed;
00029 using std::setprecision;
00030 using std::sort;
00031 using std::accumulate;
00032 using std::ofstream;
00033 using std::ifstream;
00034 using std::cerr;
00035 using std::chrono::high_resolution_clock;
00036 using std::chrono::duration;
00037 using std::stringstream;
00038 using std::ostringstream;
00039 using std::terminate;
00040
00041 // struct Studentas
00042 // {
00043 //     string vardas, pavarde;
00044 //     vector<int> pazymiai;
00045 //     int egzaminas;
00046 //     double galutinis;
00047 //     double galutinis_mediana;
00048 // };
00049
00050 class Zmogus
00051 {
00052     private:
00053         string vardas;
00054         string pavarde;
00055     public:
00056         Zmogus()
00057         {
00058             vardas="Vardas";
00059             pavarde="Pavarde";
00060         }
00061         Zmogus(string A, string B) : vardas{A}, pavarde{B} {}
00062         string getVardas() const
00063         {
00064             return vardas;
00065         }
00066         string getVardas() { return vardas; }
00067         void setVardas(const string& v) { vardas = v; }
00068         string getPavarde() const
00069         {
00070             return pavarde;
00071         }
00072         string getPavarde() { return pavarde; }
00073 
```

```

00074     string getPavarde() { return pavarde; }
00075     void setPavarde(const string& p) { pavarde = p;}
00077     Zmogus(const Zmogus & s) : vardas{s.vardas}, pavarde{s.pavarde} {}
00079     Zmogus(Zmogus && s) : vardas{s.vardas}, pavarde{s.pavarde}
00080     {
00081         s.vardas.clear();
00082         s.pavarde.clear();
00083     }
00085     Zmogus & operator=(const Zmogus& s)
00086     {
00087         if(&s == this)
00088             return *this;
00089         vardas=s.vardas;
00090         pavarde=s.pavarde;
00091         return *this;
00092     }
00094     Zmogus & operator=(Zmogus&& s)
00095     {
00096         if(&s == this)
00097             return *this;
00098         vardas=s.vardas;
00099         pavarde=s.pavarde;
00100
00101         s.vardas.clear();
00102         s.pavarde.clear();
00103         return *this;
00104     }
00105     virtual void Spausdinti() const = 0;
00107     virtual ~Zmogus()
00108     {
00109         vardas.clear();
00110         pavarde.clear();
00111     }
00112
00113 };
00114 class Studentas : public Zmogus
00115 {
00116     private:
00117         int egzaminas;
00118         double galutinis;
00119         double galutinis_mediana;
00120         vector<int>pazymiai;
00121     public:
00123         Studentas() : Zmogus()
00124         {
00125             egzaminas=0;
00126             galutinis=0;
00127             galutinis_mediana=0;
00128         }
00130         Studentas(string A, string B, vector<int>C, int D) : Zmogus(A, B), pazymiai{C}, egzaminas{D}
00131         {
00132             galutinis = Galutinis();
00133             galutinis_mediana = Galutinis_mediana();
00134         }
00135         int getEgzaminas() const
00136         {
00137             return egzaminas;
00138         }
00139         int getEgzaminas() { return egzaminas; }
00140         double getGalutinis() const
00141         {
00142             return galutinis;
00143         }
00144         double getGalutinis() { return galutinis; }
00145         double getGalutinis_mediana() const
00146         {
00147             return galutinis_mediana;
00148         }
00149         double getGalutinis_mediana() { return galutinis_mediana; }
00151         Studentas(const Studentas & s) : Zmogus(s), pazymiai{s.pazymiai}, egzaminas{s.egzaminas},
galutinis{s.galutinis}, galutinis_mediana{s.galutinis_mediana} {}
00153         Studentas(Studentas && s) : Zmogus(std::move(static_cast<Zmogus&>(s))),
pazymiai{std::move(s.pazymiai)}, egzaminas{s.egzaminas}, galutinis{s.galutinis},
galutinis_mediana{s.galutinis_mediana}
00154         {
00155             s.pazymiai.clear();
00156             s.egzaminas=0;
00157             s.galutinis=0;
00158             s.galutinis_mediana=0;
00159         }
00161         Studentas & operator=(const Studentas& s)
00162         {
00163             if(&s == this)
00164                 return *this;
00165             Zmogus::operator=(s);
00166             pazymiai.clear();
00167             pazymiai=s.pazymiai;

```

```

00168         egzaminas=s.egzaminas;
00169         galutinis=s.galutinis;
00170         galutinis_mediana=s.galutinis_mediana;
00171         return *this;
00172     }
00173     Studentas & operator=(Studentas&& s)
00174     {
00175         if(&s == this)
00176             return *this;
00177         Zmogus::operator=(std::move(static_cast<Zmogus>(s)));
00178         pazymiai.clear();
00179         pazymiai=s.pazymiai;
00180         egzaminas=s.egzaminas;
00181         galutinis=s.galutinis;
00182         galutinis_mediana=s.galutinis_mediana;
00183
00184         s.pazymiai.clear();
00185         s.egzaminas=0;
00186         s.galutinis=0;
00187         s.galutinis_mediana=0;
00188         return *this;
00189     }
00190
00191     void Spausdinti() const override
00192     {
00193         cout<<getVardas()<<" "<<getPavarde()<<endl;
00194     }
00195     friend std::istream& operator>(std::istream& in, Studentas& A)
00196     {
00197         string temp_vardas, temp_pavarde;
00198         in>>temp_vardas>>temp_pavarde;
00199         A.setVardas(temp_vardas);
00200         A.setPavarde(temp_pavarde);
00201         int x;
00202         while(in>>x)
00203         {
00204             A.pazymiai.push_back(x);
00205         }
00206         if(!A.pazymiai.empty())
00207         {
00208             A.egzaminas=A.pazymiai.back();
00209             A.pazymiai.pop_back();
00210         }
00211         A.galutinis = A.Galutinis();
00212         A.galutinis_mediana = A.Galutinis_mediana();
00213         return in;
00214     }
00215     friend std::ostream& operator<(std::ostream& out, const Studentas& A)
00216     {
00217         out<<left<<setw(20)<<A.getVardas()<<setw(20)<<A.getPavarde()<<setw(20)<<fixed<<setprecision(2)<<A.galutinis<<setw(20)<<fixed<<
00218         setprecision(2)<<A.galutinis_mediana<<endl;
00219         return out;
00220     }
00221     double Galutinis()
00222     {
00223         return (pazymiai.empty()) ? egzaminas*0.6 : accumulate(pazymiai.begin(), pazymiai.end(),
00224         0.0)/pazymiai.size()*0.4 + egzaminas*0.6;
00225     }
00226     double Galutinis_mediana()
00227     {
00228         sort(pazymiai.begin(), pazymiai.end());
00229         if(pazymiai.empty())
00230             return egzaminas*0.6;
00231         else return (pazymiai.size()%2==1) ? pazymiai[pazymiai.size()/2]*0.4+egzaminas*0.6 :
00232         (pazymiai[pazymiai.size()/2]+pazymiai[pazymiai.size()/2-1])/2.0*0.4+egzaminas*0.6;
00233     }
00234     ~Studentas()
00235     {
00236         pazymiai.clear();
00237         egzaminas=0;
00238         galutinis=0;
00239         galutinis_mediana=0;
00240     }
00241     bool operator ==(const Studentas A) const
00242     {
00243         if(getVardas()==A.getVardas() && getPavarde()== A.getPavarde() &&
00244         getEgzaminas()==A.getEgzaminas() && getGalutinis()==A.getGalutinis() &&
00245         getGalutinis_mediana()==A.getGalutinis_mediana())
00246             return true;
00247         else return false;
00248     }
00249     bool Clear()
00250     {
00251         return (getVardas().empty()) && (getPavarde().empty()) && (pazymiai.empty()) &&
00252         (egzaminas==0) && (galutinis==0) && (galutinis_mediana==0);
00253     }
00254 };
00255

```

```
00256 #endif
```

6.9 README.md File Reference

6.10 testai.cpp File Reference

```
#include <iostream>
#include "catch.hpp"
#include "mylib.h"
```

Macros

- `#define` [CATCH_CONFIG_RUNNER](#)

Functions

- [TEST_CASE](#) ("Default konstruktorius", "[Default] [konstruktorius]")
- [TEST_CASE](#) ("Konstruktorius su parametrais", "[Konstruktorius] [su] [parametrais]")
- [TEST_CASE](#) ("Destruktorius", "[Destruktorius]")
- [TEST_CASE](#) ("Kopijavimo konstruktorius", "[Kopijavimo] [konstruktorius]")
- [TEST_CASE](#) ("Kopijavimo priskyrimas", "[Kopijavimo] [priskyrimas]")
- [TEST_CASE](#) ("Move konstruktorius", "[Move] [konstruktorius]")
- [TEST_CASE](#) ("Move priskyrimas", "[Move] [priskyrimas]")
- [TEST_CASE](#) ("Ivesties metodus", "[Ivesties] [metodas]")
- [TEST_CASE](#) ("Ivesties metodus", "[Ivesties] [metodas]")
- [TEST_CASE](#) ("Getteriai", "[Getteriai]")
- `void` [testuoti](#) ()

6.10.1 Macro Definition Documentation

6.10.1.1 CATCH_CONFIG_RUNNER

```
#define CATCH_CONFIG_RUNNER
```

Definition at line 2 of file [testai.cpp](#).

6.10.2 Function Documentation

6.10.2.1 TEST_CASE() [1/10]

```
TEST_CASE (
    "Default konstruktorius" ,
    " " [Default] [konstruktorius])
```

Definition at line 5 of file [testai.cpp](#).

6.10.2.2 TEST_CASE() [2/10]

```
TEST_CASE (
    "Destruktorius" ,
    "" [Destruktorius])
```

Definition at line 15 of file [testai.cpp](#).

6.10.2.3 TEST_CASE() [3/10]

```
TEST_CASE (
    "Getteriai" ,
    "" [Getteriai])
```

Definition at line 67 of file [testai.cpp](#).

6.10.2.4 TEST_CASE() [4/10]

```
TEST_CASE (
    "Ivesties metodos" ,
    "" [Ivesties][metodas])
```

Definition at line 59 of file [testai.cpp](#).

6.10.2.5 TEST_CASE() [5/10]

```
TEST_CASE (
    "Ivesties metodos" ,
    "" [Ivesties][metodas])
```

Definition at line 50 of file [testai.cpp](#).

6.10.2.6 TEST_CASE() [6/10]

```
TEST_CASE (
    "Konstruktorius su parametrais" ,
    "" [Konstruktorius][su][parametrais])
```

Definition at line 10 of file [testai.cpp](#).

6.10.2.7 TEST_CASE() [7/10]

```
TEST_CASE (
    "Kopijavimo konstruktorius" ,
    "" [Kopijavimo][konstruktorius])
```

Definition at line 22 of file [testai.cpp](#).

6.10.2.8 TEST_CASE() [8/10]

```
TEST_CASE (
    "Kopijavimo priskyrimas" ,
    " " [Kopijavimo][priskyrimas])
```

Definition at line 28 of file [testai.cpp](#).

6.10.2.9 TEST_CASE() [9/10]

```
TEST_CASE (
    "Move konstruktorius" ,
    " " [Move][konstruktorius])
```

Definition at line 35 of file [testai.cpp](#).

6.10.2.10 TEST_CASE() [10/10]

```
TEST_CASE (
    "Move priskyrimas" ,
    " " [Move][priskyrimas])
```

Definition at line 42 of file [testai.cpp](#).

6.10.2.11 testuoti()

```
void testuoti ()
```

Definition at line 77 of file [testai.cpp](#).

6.11 testai.cpp

[Go to the documentation of this file.](#)

```
00001 #include <iostream>
00002 #define CATCH_CONFIG_RUNNER
00003 #include "catch.hpp"
00004 #include "mylib.h"
00005 TEST_CASE("Default konstruktorius", "[Default] [konstruktorius]")
00006 {
00007     Studentas stud;
00008     REQUIRE(stud==Studentas("Vardas", "Pavarde", { }, 0));
00009 }
00010 TEST_CASE("Konstruktorius su parametrais", "[Konstruktorius] [su] [parametrais]")
00011 {
00012     Studentas Stud("Vardenis", "Pavardenis", {5, 8, 3}, 9);
00013     REQUIRE(Stud==Studentas("Vardenis", "Pavardenis", {5, 8, 3}, 9));
00014 }
00015 TEST_CASE("Destruktorius", "[Destruktorius]")
00016 {
00017     {
00018         Studentas s("Vardenis", "Pavardenis", {5,8,3}, 9);
00019     }
00020     SUCCEED("Destruktorius: [OK]");
00021 }
00022 TEST_CASE("Kopijavimo konstruktorius", "[Kopijavimo] [konstruktorius]")
00023 {
00024     Studentas Stud("Vardenis", "Pavardenis", {5, 8, 3}, 9);
00025     Studentas stud(Stud);
```



```

00026     REQUIRE(stud==Stud);
00027 }
00028 TEST_CASE("Kopijavimo priskyrimas", "[Kopijavimo] [priskyrimas]")
00029 {
00030     Studentas Stud("Vardenis", "Pavardenis", {5, 8, 3}, 9);
00031     Studentas stud;
00032     stud=Stud;
00033     REQUIRE(stud==Stud);
00034 }
00035 TEST_CASE("Move konstruktorius", "[Move] [konstruktorius]")
00036 {
00037     Studentas Stud("Vardenis", "Pavardenis", {5, 8, 3}, 9);
00038     Studentas stud(std::move(Stud));
00039     REQUIRE(stud==Studentas("Vardenis", "Pavardenis", {5, 8, 3}, 9));
00040     REQUIRE(Stud.Clear()==true);
00041 }
00042 TEST_CASE("Move priskyrimas", "[Move] [priskyrimas]")
00043 {
00044     Studentas Stud("Vardenis", "Pavardenis", {5, 8, 3}, 9);
00045     Studentas stud;
00046     stud=std::move(Stud);
00047     REQUIRE(stud==Studentas("Vardenis", "Pavardenis", {5, 8, 3}, 9));
00048     REQUIRE(Stud.Clear()==true);
00049 }
00050 TEST_CASE("Ivesties metodos", "[Ivesties] [metodos]")
00051 {
00052     std::istringstream in("Jonas Jonaitis 10 9 8 7");
00053     Studentas s;
00054     in >> s;
00055     REQUIRE(s.getVardas()=="Jonas");
00056     REQUIRE(s.getPavarde()=="Jonaitis");
00057     REQUIRE(s.getEgzaminas()==7);
00058 }
00059 TEST_CASE("Isvesties metodos", "[Isvesties] [metodos]")
00060 {
00061     Studentas sl("Jonas", "Jonaitis", {10, 9, 8}, 7);
00062     std::ostringstream out;
00063     out << sl;
00064     REQUIRE(out.str().find("Jonas") != string::npos);
00065     REQUIRE(out.str().find("Jonaitis") != string::npos);
00066 }
00067 TEST_CASE("Getteriai", "[Getteriai]")
00068 {
00069     Studentas Stud("Vardenis", "Pavardenis", {5, 8, 3}, 9);
00070     REQUIRE(Stud.getVardas()=="Vardenis");
00071     REQUIRE(Stud.getPavarde()=="Pavardenis");
00072     REQUIRE(Stud.getEgzaminas()==9);
00073     REQUIRE(Stud.getGalutinis()==Stud.Galutinis());
00074     REQUIRE(Stud.getGalutinis_mediana()==Stud.Galutinis_mediana());
00075 }
00076
00077 void testuoti()
00078 {
00079     int argc=1;
00080     char* argv[]={(char*)"testai"};
00081     Catch::Session().run(argc, argv);
00082 }

```


Index

- ~Studentas
 - Studentas, [13](#)
- ~Zmogus
 - Zmogus, [17](#)
- CATCH_CONFIG_RUNNER
 - testai.cpp, [40](#)
- Clear
 - Studentas, [13](#)
- dalinimas_i_kategorijas_1
 - funkcijos.h, [28](#)
- dalinimas_i_kategorijas_2
 - funkcijos.h, [28](#)
- dalinimas_i_kategorijas_3
 - funkcijos.h, [28](#)
- dar_vienas_testas
 - funkcijos.h, [28](#)
- did_gal_med
 - funkcijos.cpp, [21](#)
 - funkcijos.h, [28](#)
- did_gal_vid
 - funkcijos.cpp, [21](#)
 - funkcijos.h, [29](#)
- did_pav
 - funkcijos.cpp, [22](#)
 - funkcijos.h, [29](#)
- did_var
 - funkcijos.cpp, [22](#)
 - funkcijos.h, [29](#)
- failu_generavimas
 - funkcijos.cpp, [22](#)
 - funkcijos.h, [29](#)
- funkcijos.cpp, [21](#)
 - did_gal_med, [21](#)
 - did_gal_vid, [21](#)
 - did_pav, [22](#)
 - did_var, [22](#)
 - failu_generavimas, [22](#)
 - maz_gal_med, [22](#)
 - maz_gal_vid, [22](#)
 - maz_pav, [22](#)
 - maz_var, [23](#)
 - raides, [23](#)
 - rikiuoti, [23](#)
 - testas_1, [23](#)
- funkcijos.h, [27](#)
 - dalinimas_i_kategorijas_1, [28](#)
 - dalinimas_i_kategorijas_2, [28](#)
 - dalinimas_i_kategorijas_3, [28](#)
 - dar_vienas_testas, [28](#)
 - did_gal_med, [28](#)
 - did_gal_vid, [29](#)
 - did_pav, [29](#)
 - did_var, [29](#)
 - failu_generavimas, [29](#)
 - generuoti_viska, [29](#)
 - konstruktoriu_testas, [29](#)
 - maz_gal_med, [29](#)
 - maz_gal_vid, [30](#)
 - maz_pav, [30](#)
 - maz_var, [30](#)
 - pasirinkimas, [30](#)
 - pazymiu_generavimas, [30](#)
 - raides, [30](#)
 - rikiavimas, [31](#)
 - rikiuoti, [31](#)
 - skaiciavimai, [31](#)
 - skaitymas, [31](#)
 - skaitymas_is_failo, [32](#)
 - spausdinimas, [32](#)
 - spausdinimas_i_ekrana, [32](#)
 - spausdinimas_i_faila, [32](#)
 - testas_1, [32](#)
 - testas_2, [32](#)
 - testas_3, [32](#)
 - testuoti, [33](#)
 - vector_list_deque, [33](#)
- Galutinis
 - Studentas, [13](#)
- Galutinis_mediana
 - Studentas, [14](#)
- generuoti_viska
 - funkcijos.h, [29](#)
- getEgzaminas
 - Studentas, [14](#)
- getGalutinis
 - Studentas, [14](#)
- getGalutinis_mediana
 - Studentas, [14](#)
- getPavarde
 - Zmogus, [18](#)
- getVardas
 - Zmogus, [18](#)
- konstruktoriu_testas
 - funkcijos.h, [29](#)

- main
 - main.cpp, 34
- main.cpp, 34
 - main, 34
- maz_gal_med
 - funkcijos.cpp, 22
 - funkcijos.h, 29
- maz_gal_vid
 - funkcijos.cpp, 22
 - funkcijos.h, 30
- maz_pav
 - funkcijos.cpp, 22
 - funkcijos.h, 30
- maz_var
 - funkcijos.cpp, 23
 - funkcijos.h, 30
- mylib.h, 36
- Objektinis-programavimas, 1
- operator<<
 - Studentas, 15
- operator>>
 - Studentas, 15
- operator=
 - Studentas, 15
 - Zmogus, 18
- operator==
 - Studentas, 15
- pasirinkimas
 - funkcijos.h, 30
- pazymiu_generavimas
 - funkcijos.h, 30
- raides
 - funkcijos.cpp, 23
 - funkcijos.h, 30
- README.md, 40
- rikiavimas
 - funkcijos.h, 31
- rikiuoti
 - funkcijos.cpp, 23
 - funkcijos.h, 31
- setPavarde
 - Zmogus, 18
- setVardas
 - Zmogus, 19
- skaiciavimai
 - funkcijos.h, 31
- skaitymas
 - funkcijos.h, 31
- skaitymas_is_failo
 - funkcijos.h, 32
- spausdinimas
 - funkcijos.h, 32
- spausdinimas_i_ekrana
 - funkcijos.h, 32
- spausdinimas_i_faila
 - funkcijos.h, 32
- Spausdinti
 - Studentas, 15
 - Zmogus, 19
- Studentas, 11
 - ~Studentas, 13
 - Clear, 13
 - Galutinis, 13
 - Galutinis_mediana, 14
 - getEgzaminas, 14
 - getGalutinis, 14
 - getGalutinis_mediana, 14
 - operator<<, 15
 - operator>>, 15
 - operator=, 15
 - operator==, 15
 - Spausdinti, 15
 - Studentas, 12, 13
- TEST_CASE
 - testai.cpp, 40–42
- testai.cpp, 40
 - CATCH_CONFIG_RUNNER, 40
 - TEST_CASE, 40–42
 - testuoti, 42
- testas_1
 - funkcijos.cpp, 23
 - funkcijos.h, 32
- testas_2
 - funkcijos.h, 32
- testas_3
 - funkcijos.h, 32
- testuoti
 - funkcijos.h, 33
 - testai.cpp, 42
- vector_list_deque
 - funkcijos.h, 33
- Zmogus, 16
 - ~Zmogus, 17
 - getPavarde, 18
 - getVardas, 18
 - operator=, 18
 - setPavarde, 18
 - setVardas, 19
 - Spausdinti, 19
 - Zmogus, 17